

Rapport de TP - Day 2

Programmation Orientee Objet (OOP) & API REST avec FastAPI

Date de rendu :	30/06/2026 10:27
Etudiant :	Amine (M2)
Validation globale :	TOUTES LES ETAPES ET ENDPOINTS CONFORMES & VALIDES

1. Refactoring Oriente Objet (Lab 2.1)

Ce module applique les concepts de programmation orientee objet (classes, dataclasses, exceptions personnalisées) et de programmation asynchrone (async/await, asyncio.gather) pour refactoriser notre outil de monitoring de serveurs.

Etape 1.1 : Dataclass Server

Modelisation du conteneur de donnees Server avec une structure immutable par default pour les tags et une methode dynamique calculant l'URL de base.

```
@dataclass
class Server:
    id: int
    name: str
    host: str
    port: int
    status: str = 'unknown'
    tags: list[str] = field(default_factory=list)

    def base_url(self) -> str:
        protocol = 'https' if self.port == 443 else 'http'
        return f"{protocol}://{self.host}:{self.port}"
```

Etape 1.2 : ConfigLoader avec Gestion de Fichiers & Logging

La classe ConfigLoader se charge de charger et parser le fichier de configuration JSON. Elle leve une exception dediee ConfigError et loggue les evenements importants.

```
class ConfigError(ValueError):
    """Raised when config loading fails."""
    pass

class ConfigLoader:
    """Loads server configuration from a JSON file."""
    def __init__(self, path: str):
        self.path = pathlib.Path(path)

    def load(self) -> list[Server]:
        try:
            raw = self.path.read_text(encoding='utf-8')
            data = json.loads(raw)
            servers = []
            for idx, item in enumerate(data, start=1):
                servers.append(Server(
                    id=idx, name=item['name'], host=item['host'],
                    port=item['port'], tags=item.get('tags', [])
                ))
```

```
        logger.info("Successfully loaded %d servers", len(servers))
        return servers
    except FileNotFoundError as e:
        logger.error("Config file not found")
        raise ConfigError(f"File not found") from e
    except json.JSONDecodeError as e:
        logger.error("Invalid JSON format")
        raise ConfigError(f"Invalid JSON") from e
```

Etape 1.3 : HealthChecker Asynchrone (Concurrency)

La classe HealthChecker utilise `httpx.AsyncClient` pour envoyer des requetes paralleles. `asyncio.gather` permet de verifier tous les serveurs simultanement plutot que sequentiellement.

```
class HealthChecker:
    def __init__(self, timeout: float = 5.0, degraded_threshold_ms: float = 500.0):
        self.timeout = timeout
        self.degraded_threshold_ms = degraded_threshold_ms

    async def check(self, server: Server) -> Server:
        path = "/status/200" if "httpbin.org" in server.host else "/health"
        url = f"{server.base_url()}{path}"
        start = time.time()
        try:
            async with httpx.AsyncClient(timeout=self.timeout) as client:
                resp = await client.get(url)
                elapsed_ms = (time.time() - start) * 1000
                if resp.status_code == 200:
                    server.status = "UP" if elapsed_ms <= self.degraded_threshold_ms else "DEGRADED"
                else:
                    server.status = "DEGRADED"
        except Exception:
            server.status = "DOWN"
        return server

    async def check_all(self, servers: list[Server]) -> list[Server]:
        tasks = [self.check(s) for s in servers]
        return await asyncio.gather(*tasks)
```

Verification et Sorties de l'Execution :

OOP Test failed: No module named 'httpx'

2. API REST CRUD avec FastAPI (Lab 2.2)

Construction d'une API web complete permettant de gerer dynamiquement les serveurs enregistres en memoire. L'API supporte l'enregistrement (POST), le listage avec filtrage par statut (GET), la consultation unitaire (GET), la suppression (DELETE) et le declenchement manuel d'un check de sante (POST).

Schemas Pydantic (Validation & Serialization)

```
class ServerIn(BaseModel):
    name: str
    host: str
    port: int = Field(default=8080, ge=1, le=65535)
    tags: list[str] = []

class ServerOut(BaseModel):
    id: int
    name: str
    host: str
    port: int
    status: str
    tags: list[str] = []
    model_config = {"from_attributes": True}
```

Implementation des Endpoints CRUD & Check

```
@app.get("/servers", response_model=list[ServerOut])
async def list_servers(status: str | None = None):
    servers = list(_store.values())
    if status:
        return [s for s in servers if s.status.upper() == status.upper()]
    return servers

@app.get("/servers/{server_id}", response_model=ServerOut)
async def get_server(server_id: int):
    if server_id not in _store:
        raise HTTPException(status_code=404, detail="Server not found")
    return _store[server_id]

@app.delete("/servers/{server_id}", status_code=204)
async def delete_server(server_id: int):
    if server_id not in _store:
        raise HTTPException(status_code=404, detail="Server not found")
    del _store[server_id]
    return

@app.post("/servers/{server_id}/check", response_model=ServerOut)
async def trigger_check(server_id: int):
    if server_id not in _store:
        raise HTTPException(404, "Server not found")
    server = _store[server_id]
    try:
        async with httpx.AsyncClient(timeout=5.0) as client:
            path = "/status/200" if "httpbin" in server.host else "/health"
            resp = await client.get(f"{server.base_url()}{path}")
            server.status = "UP" if resp.status_code == 200 else "DEGRADED"
    except Exception:
        server.status = "DOWN"
    return server
```

Verification et Sorties de l'Execution :

API Test failed: No module named 'fastapi'